

THE HONG KONG POLYTECHNIC UNIVERSITY
香港理工大學

Opening Minds • Shaping the Future
啟迪思維 • 成就未來

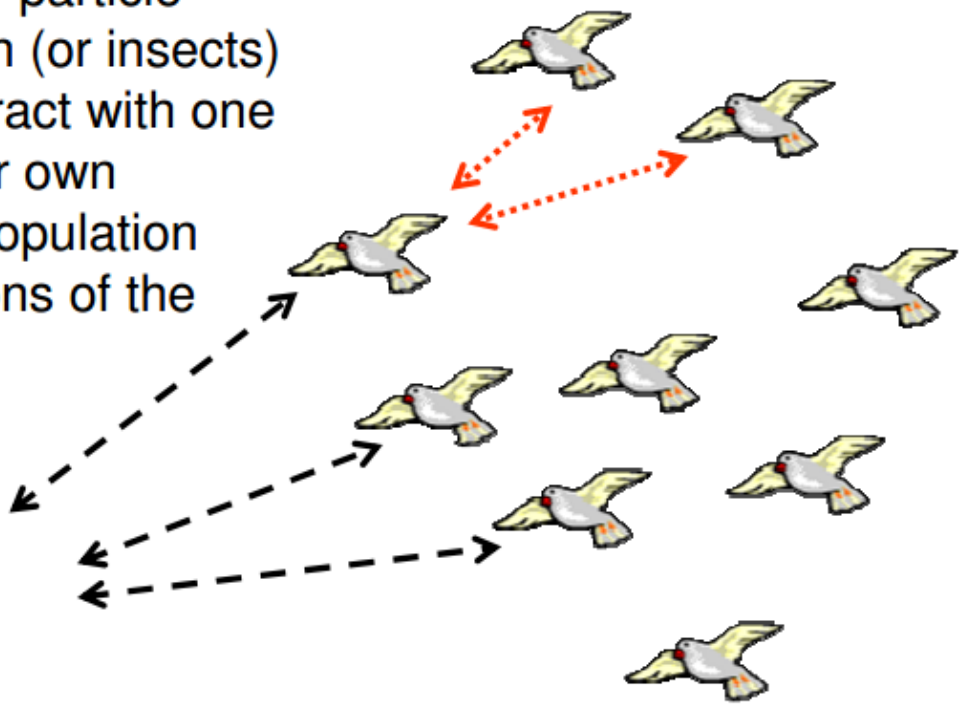
DSAI6707 Advanced Computational Intelligence

Sem 1, 2026/2027

Particle Swarm Optimization



As described by the inventers James Kennedy and Russell Eberhart, "particle swarm algorithm imitates human (or insects) social behavior. Individuals interact with one another while learning from their own experience, and gradually the population members move into better regions of the problem space".





Why named as "Particle", not "points"? Both Kennedy and Eberhart felt that velocities and accelerations are more appropriately applied to particles.

Classical PSO Algorithm

- PSO is a robust stochastic optimization technique based on the movement and intelligence of swarms
- PSO applies the concept of social interaction to problem solving
- It was developed in 1995 by James Kennedy (social-psychologist) and Russell Eberhart (electrical engineer)
- It uses a number of agents (particles) that constitute a swarm moving around in the search space looking for the best solution
- Each particle is treated as a point in a N-dimensional space which adjusts its "flying" according to its own flying experience as well as the flying experience of other particles
- Each particle keeps track of its coordinates in the solution space which are associated with the best solution (fitness) that has achieved so far by that particle. This value is called personal best, ***pbest***

- Another best value that is tracked by the PSO is the best value obtained so far by any particle in the neighborhood of that particle. This value is called ***gbest***
- The basic concept of PSO lies in accelerating each particle toward its *pbest* and the *gbest* locations, with a random weighted acceleration at each time step

$$V_{id} = w \times V_{id} + c_1 \times Rand() \times (p_{id} - x_{id}) + c_2 \times Rand() \times (g_d - x_{id}) \quad (1)$$

$$x_{id} = x_{id} + V_{id} \quad (2)$$

- V_{id} : Velocity of each particle in each dimension
  - i: Particle
  - d: Dimension
- w : Inertia Weight
- c_1、c_2 : Constants
- *Rand*() : Random
- p_{id} : Best position of each particle
- g_d : Best position of swarm
- x_{id} : Current position of each particle in each dimension

Algorithm 1 Classical PSO.

1: Initialize a population array of particles with random positions and velocities on D dimensions in the problem space.
2: **loop**
3: For each particle, evaluate the desired optimization fitness function in D variables.
4: Compare particle's fitness evaluation with its personal best fitness $pbest^i$. If current value is better than $pbest^i$, then set $pbest^i$ equal to the current value, and y^i equal to the current location x^i in D–dimensional space.
5: Identify the particle in the neighbourhood with the best success so far, and assign its position to the variable $\hat{y}$.
6: Change the velocity and position of the particle according to the following equations:

$$v_{t+1}^i = wv_t^i + \phi_1 \otimes (y^i - x_t^i) + \phi_2 \otimes (\hat{y} - x_t^i) \quad (1)$$

$$x_{t+1}^i = x_t^i + v_{t+1}^i \quad (2)$$

7: If a criterion is met, exit loop.
8: **end loop**

Note: ϕ_i represents a vector of random numbers uniformly distributed in $[0, c_i]$ and $\otimes$ is component-wise multiplication.

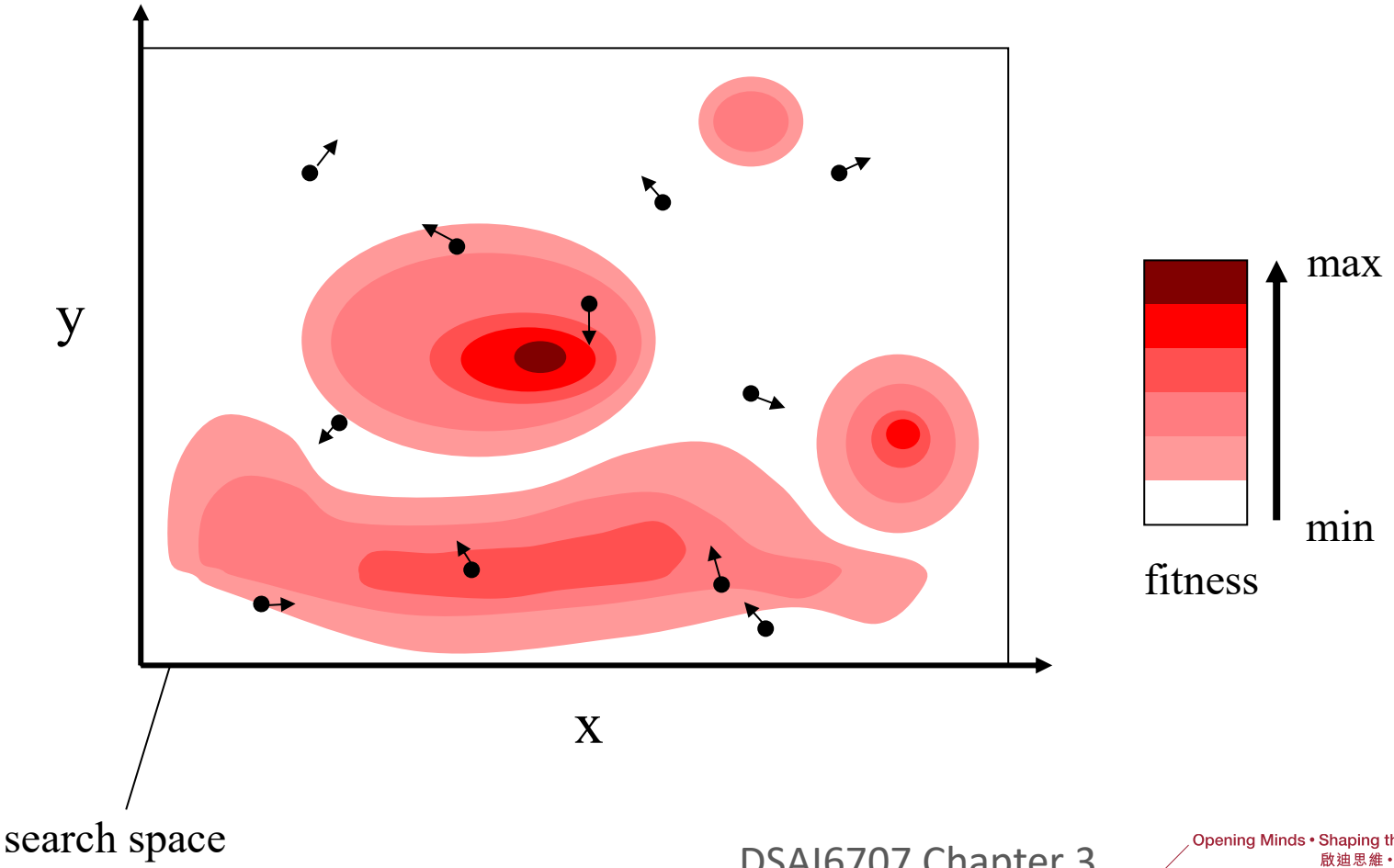
Classical PSO

- The inertia weight *w* is often chosen between [0.5, 1.5].
- A large inertia weight (*w*) facilitates a global search while a small inertia weight facilitates a local search.
- By linearly decreasing the inertia weight from a relatively large value to a small value through the course of the PSO run gives better PSO performance as compared with fixed inertia weight settings.

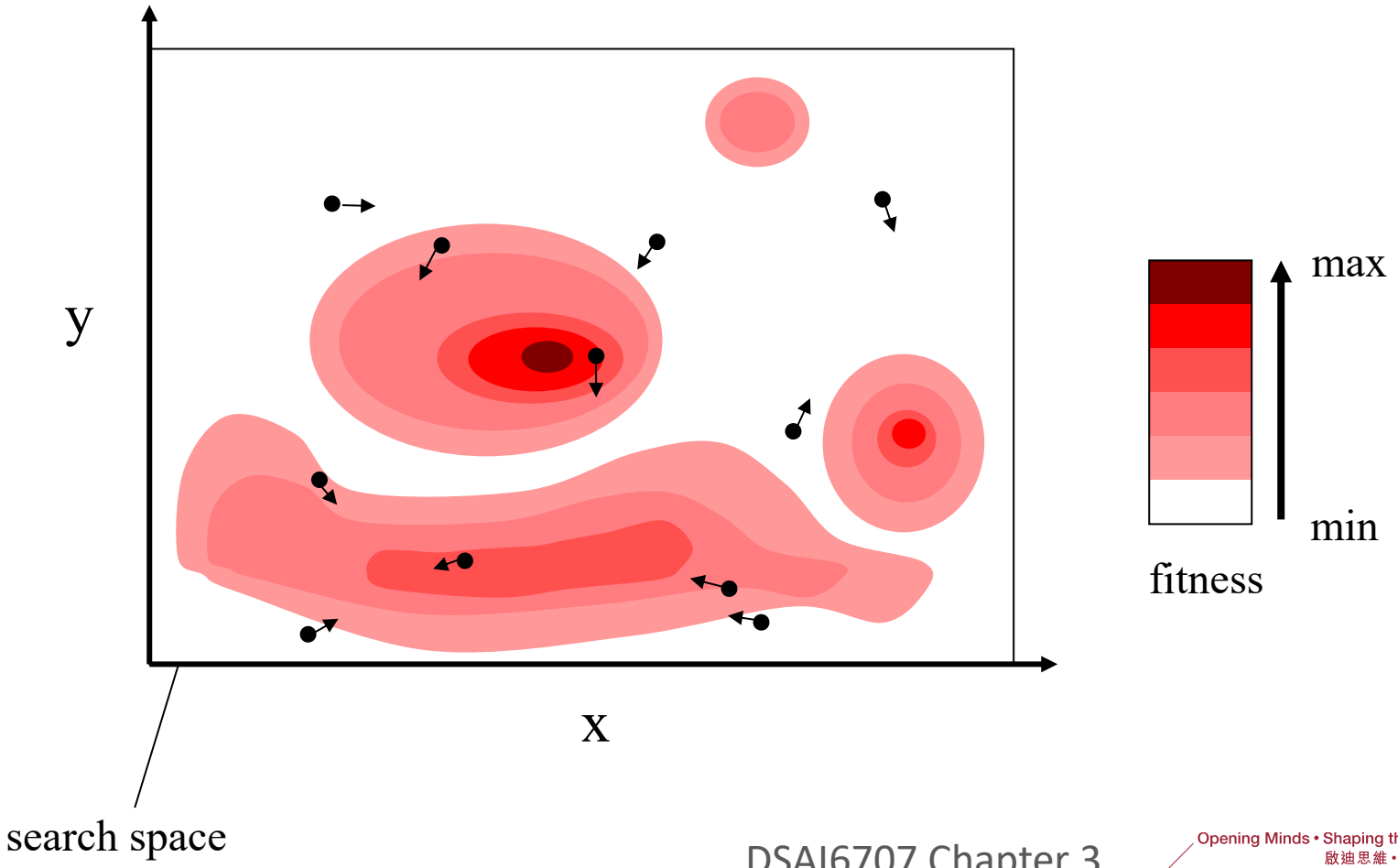
  Larger w ---------- greater global search ability

  Smaller w ---------- greater local search ability.
- The interval of *c*1 and *c*2 is [0, 4.0]. If *c*2 is higher than *c*1, the social information would influence the particle more. If *c*1 is higher than *c*2, the particle would be affected more by its personal best rather than the global best.
- Rand() is a random number between [0, 1].

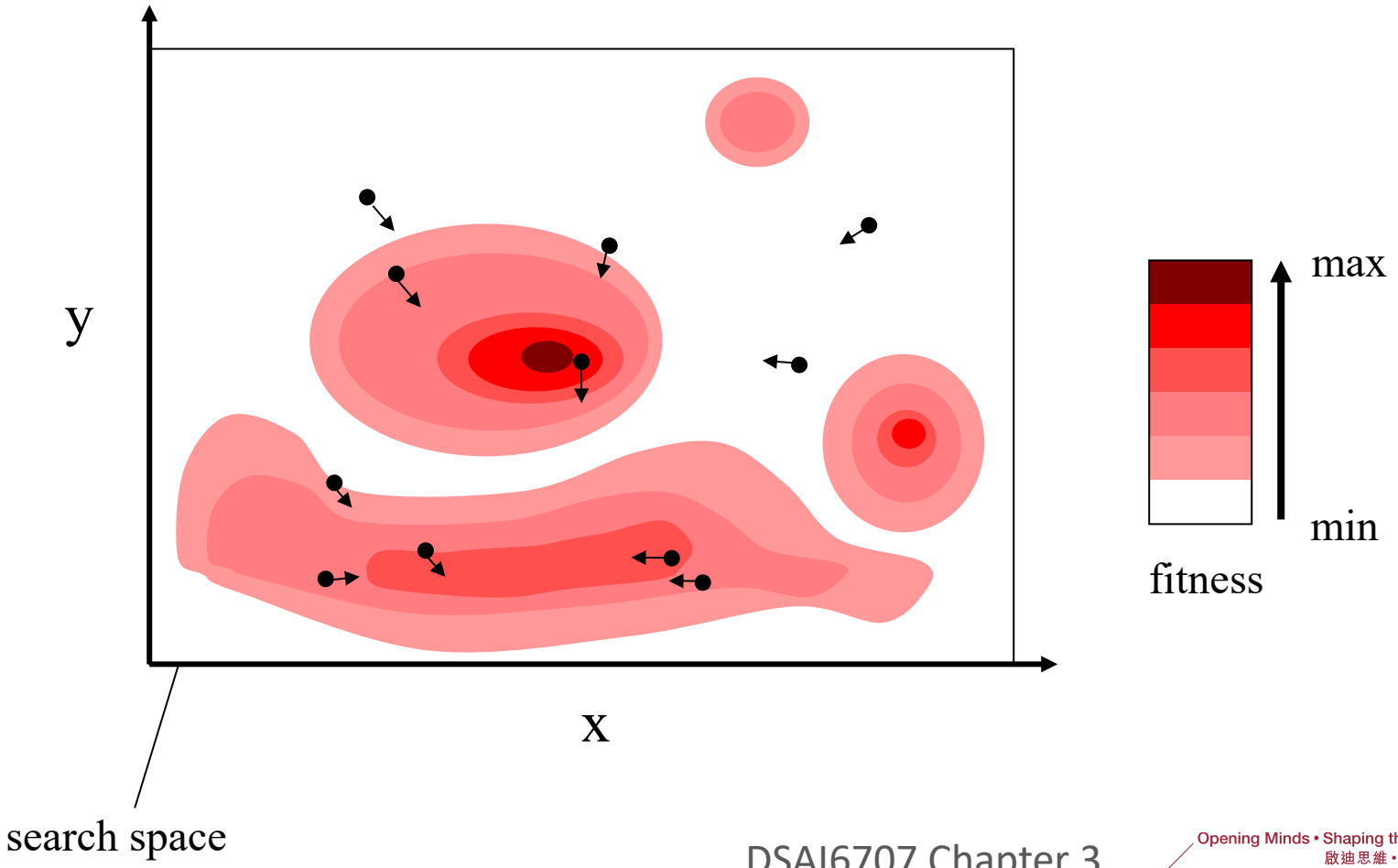
Simulation



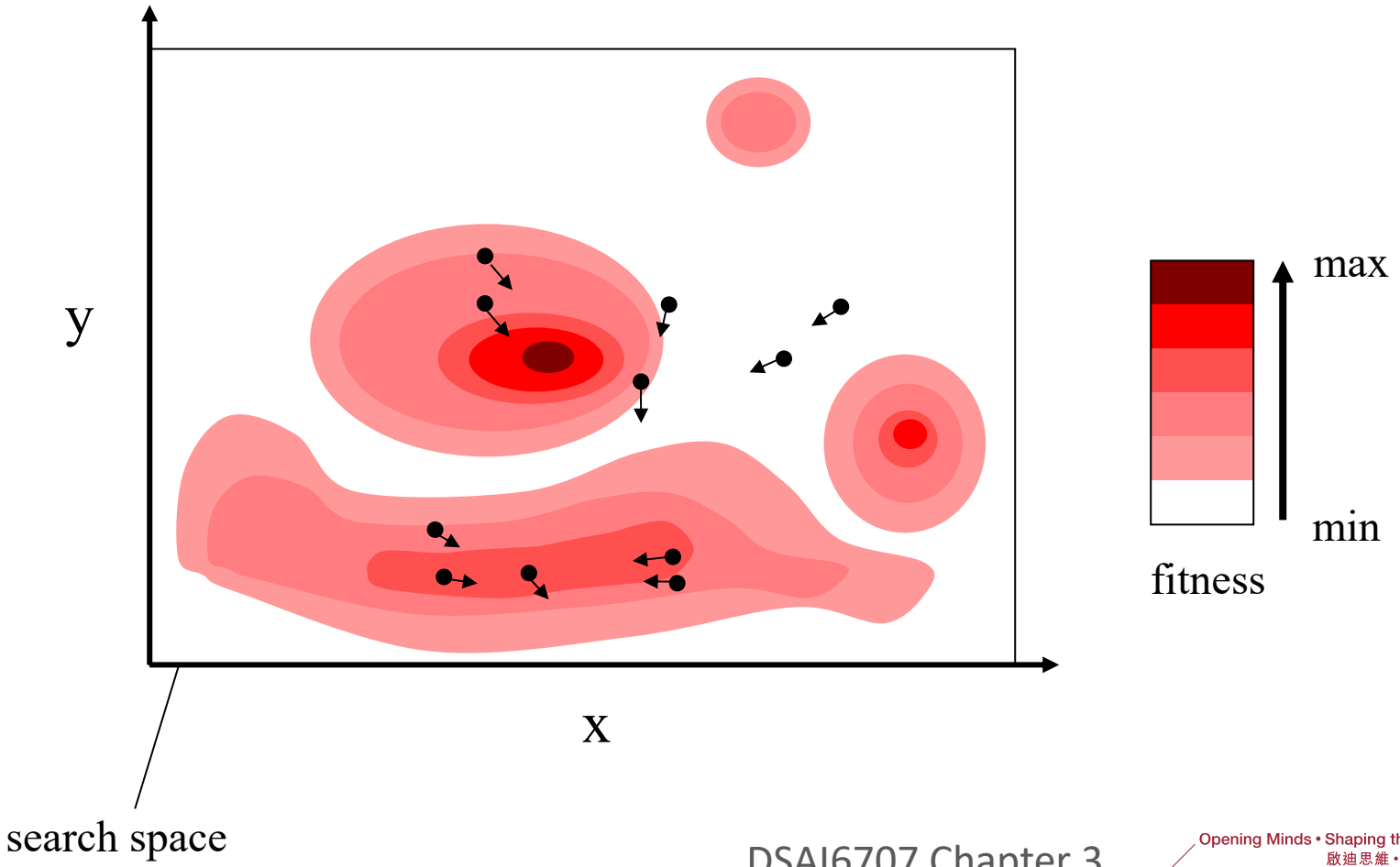
Simulation



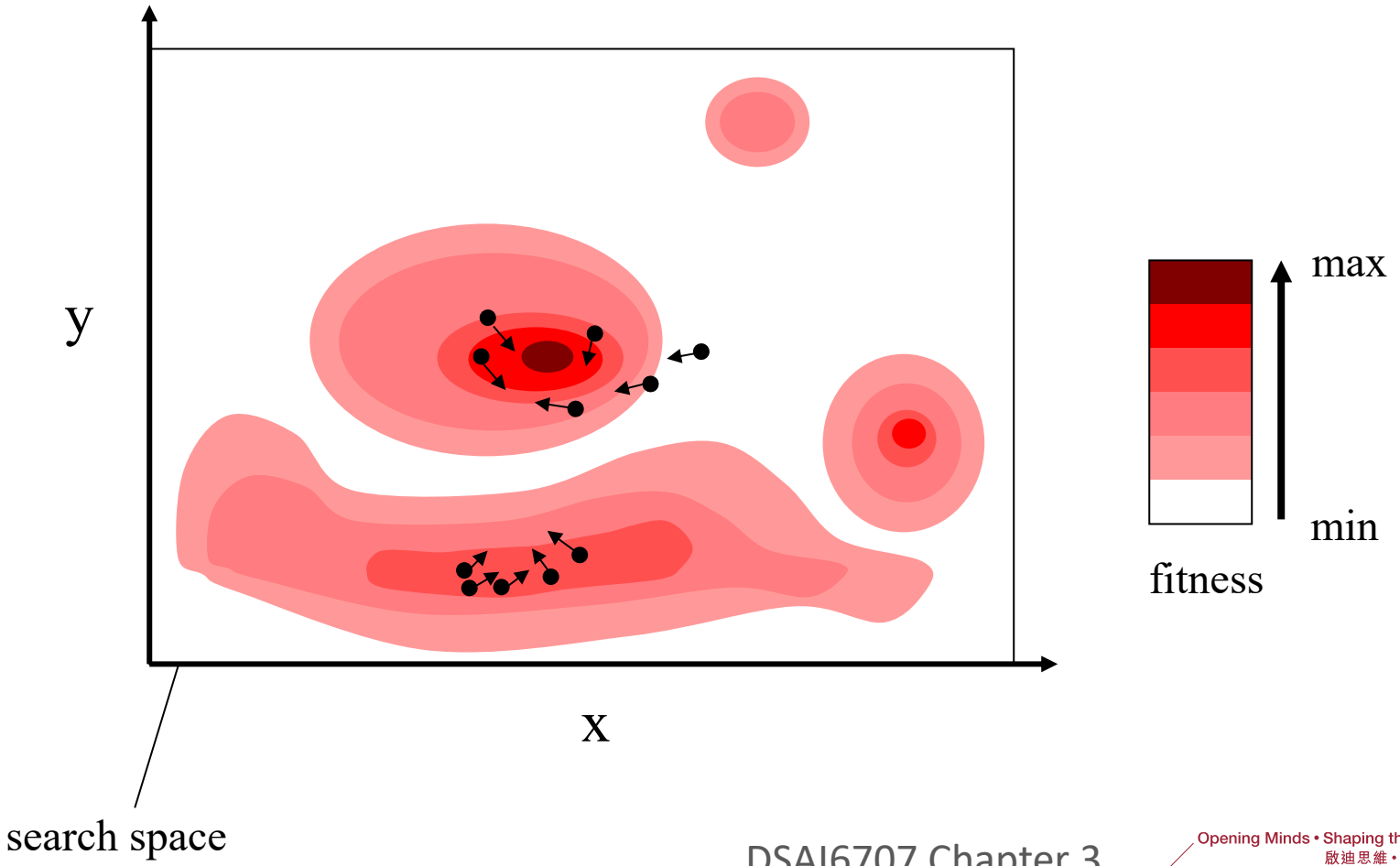
Simulation



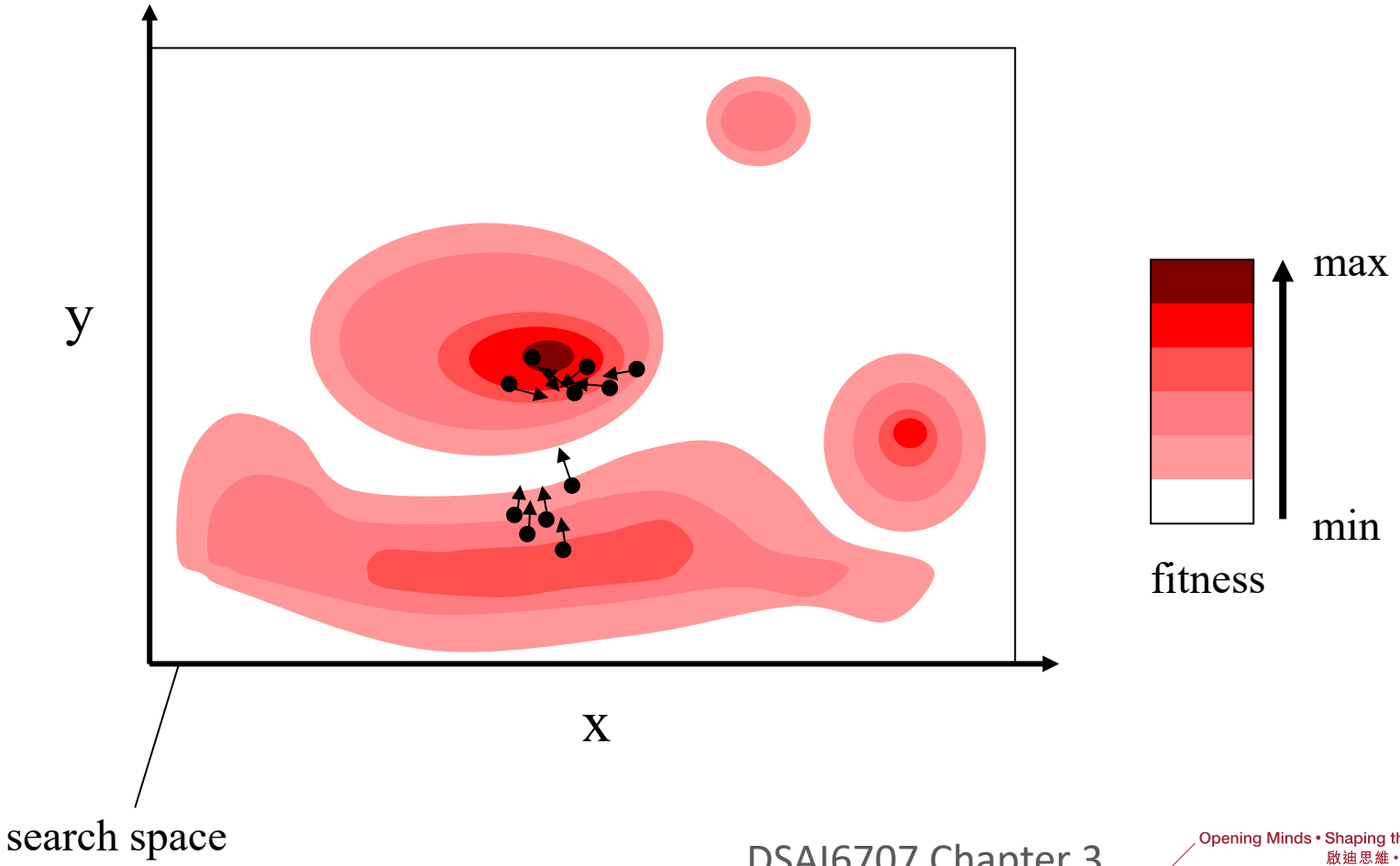
Simulation



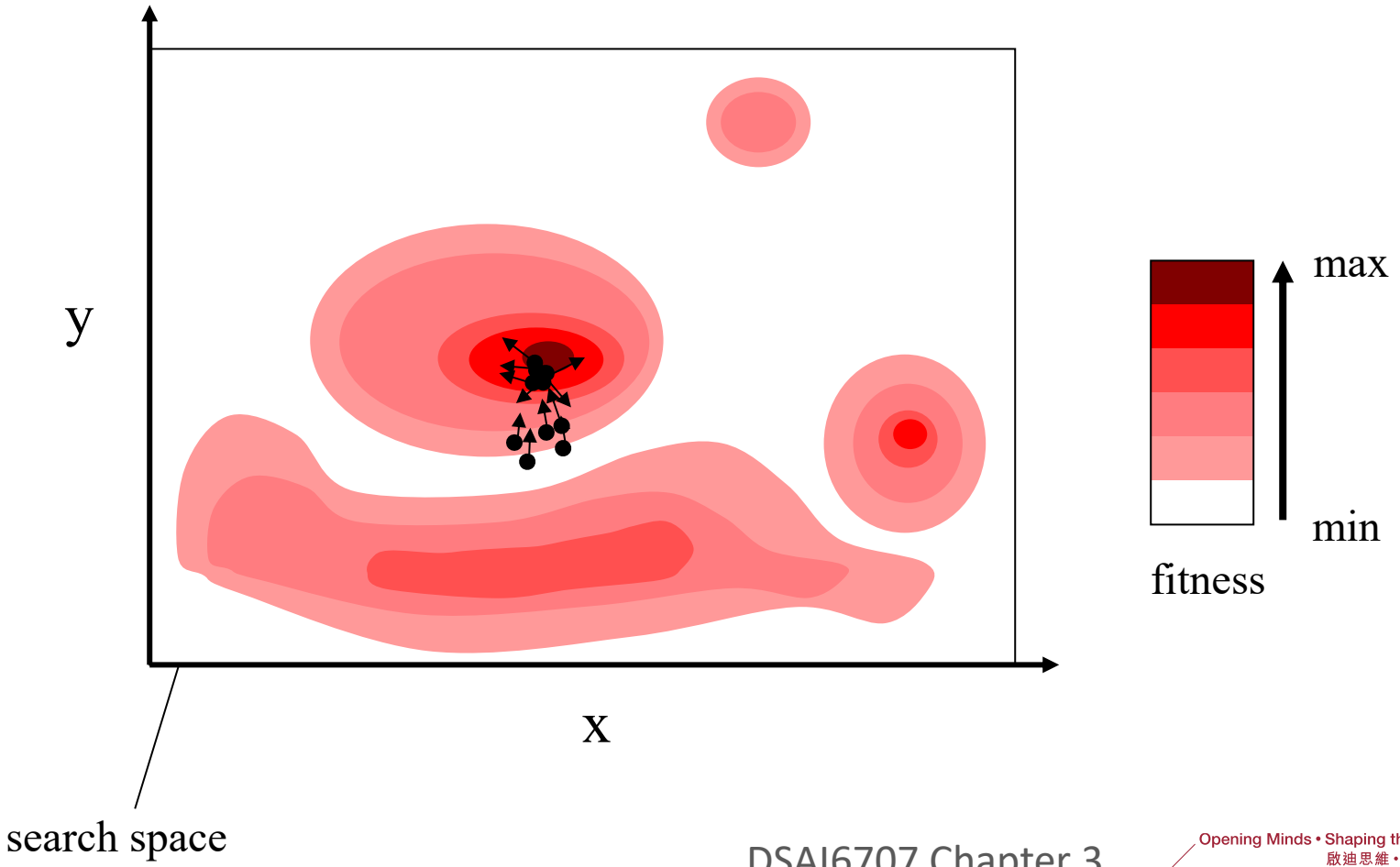
Simulation



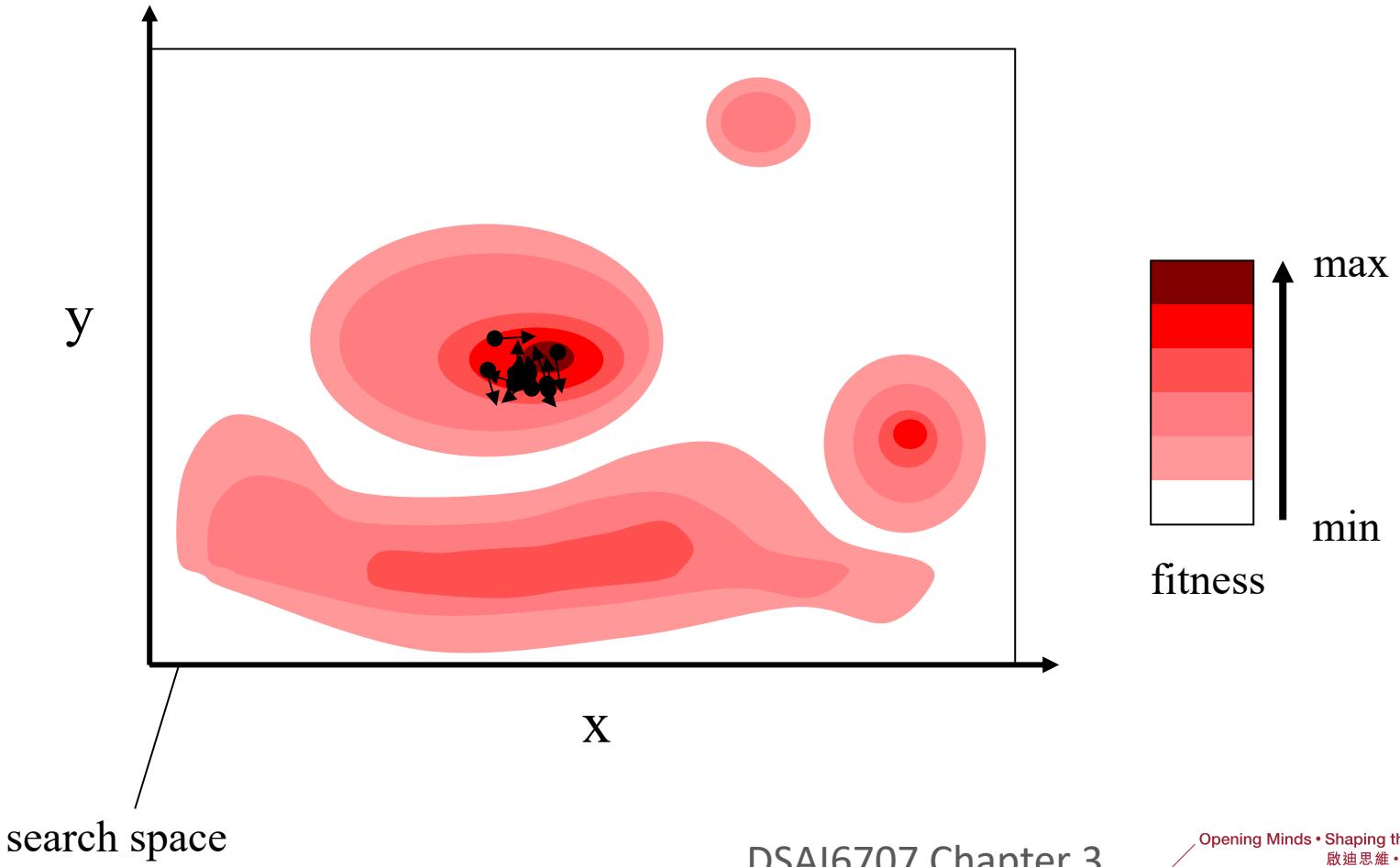
Simulation



Simulation



Simulation



Comparison with other evolutionary computation techniques:

- Unlike in genetic algorithms, evolutionary programming and evolutionary strategies, in PSO, there is no selection operation
- All particles in PSO are kept as members of the population through the course of the run.
- No need of crossover or mutation operations in PSO
- In PSO, balance between the global and local search can be adjusted through the inertial weight factor (*w*).

Differential Evolution

- Proposed by Price and Storn in 1995.
- Considered as one of the most powerful evolutionary algorithms for real number function optimization.
- Differential Evolution -- a simple and efficient adaptive scheme for global optimization over continuous spaces, 1995.
- Finished 3rd at the First International Contest on Evolutionary Computation, Nagoya, 1996.
- Outperformed GA and PSO on a 34-function test suite (Vesterstrom & Thomsen, 2004).
- Continually exhibited remarkable performance in competitions on different kinds of optimization problems like dynamic, multi-objective, constrained and multi-modal problems held under *IEEE Congress on Evolutionary Computation* (CEC) conference series.

Background

- DE's Main Idea:
  - Generate trial vectors (v) using the following formula:

$$\boldsymbol{v_i = x_{r1} + F \cdot (x_{r2} - x_{r3})}$$

where $\boldsymbol{x_{r1}}$, $\boldsymbol{x_{r2}}$, $\boldsymbol{x_{r3}}$ are randomly selected from the current population and F is a parameter called scaling factor.

  - Less parameters to be tuned.

DE Algorithm

- Initialize the population $\mathbf{X}$ with uniform random values
- Determine quality of members of $\mathbf{X}$ based on fitness function
- **repeat**
  - **For each** parent $\vec{\mathbf{X}}_c$ **in** $\mathbf{X}$
    - Randomly select a difference pair $\vec{\mathbf{X}}_a$ and $\vec{\mathbf{X}}_b$
    - Calculate $\vec{\mathbf{X}}'_c = \vec{\mathbf{X}}_c + F \times \left(\vec{\mathbf{X}}_a - \vec{\mathbf{X}}_b\right)$
    - Calculate $\vec{\mathbf{X}}''_c = \text{Crossover}\left(\vec{\mathbf{X}}'_c, \vec{\mathbf{X}}_c\right)$
    - Determine the quality of $\vec{\mathbf{X}}''_c$
    - **if** $\vec{\mathbf{X}}''_c$ is better than $\vec{\mathbf{X}}_c$
      - **then** $\vec{\mathbf{X}}''_c$ goes in new population $\mathbf{X}'$
      - **else** $\vec{\mathbf{X}}_c$ goes in new population $\mathbf{X}'$
  - **endfor**
  - $\mathbf{X}'$ replaces $\mathbf{X}$
- **until** population converged or generation limit

Different DE Mutation Schemes

"DE/rand/1": $\vec{V}_i(t) = \vec{X}_{r_1^i}(t) + F \cdot (\vec{X}_{r_2^i}(t) - \vec{X}_{r_3^i}(t))$.

"DE/best/1": $\vec{V}_i(t) = \vec{X}_{best}(t) + F.(\vec{X}_{r_1^i}(t) - \vec{X}_{r_2^i}(t))$.

"DE/target-to-best/1": $\vec{V}_i(t) = \vec{X}_i(t) + F.(\vec{X}_{best}(t) - \vec{X}_i(t)) + F.(\vec{X}_{r_1^i}(t) - \vec{X}_{r_2^i}(t))$,

"DE/best/2": $\vec{V}_i(t) = \vec{X}_{best}(t) + F.(\vec{X}_{r_1^i}(t) - \vec{X}_{r_2^i}(t)) + F.(\vec{X}_{r_3^i}(t) - \vec{X}_{r_4^i}(t))$.

"DE/rand/2": $\vec{V}_i(t) = \vec{X}_{r_1^i}(t) + F_1.(\vec{X}_{r_2^i}(t) - \vec{X}_{r_3^i}(t)) + F_2.(\vec{X}_{r_4^i}(t) - \vec{X}_{r_5^i}(t))$.

The general convention used for naming the various mutation strategies is DE/*x*/*y*/*z*, where DE stands for Differential Evolution, *x* represents a string denoting the vector to be perturbed, *y* is the number of difference vectors considered for perturbation of *x*, and *z* stands for the type of crossover being used.

Memetic algorithms (MA)

Overview of the development in evolutionary computation:

- Evolutionary programming (1960s, Fogel)
- Genetic algorithm (1970s, Rechenberg)
- Evolutionary strategies (1970s, Holland)
- Evolutionary algorithms (1990s, Goldberg)
- Genetic programming (1990s, Koza)
- **Memetic algorithms** (1990s), *Also known as…*
  - Cultural algorithms
  - Hybrid GA
  - Baldwinian GA
  - Larmarckian GA
  - Etc…(But the concepts are the same!)

Memetic algorithms

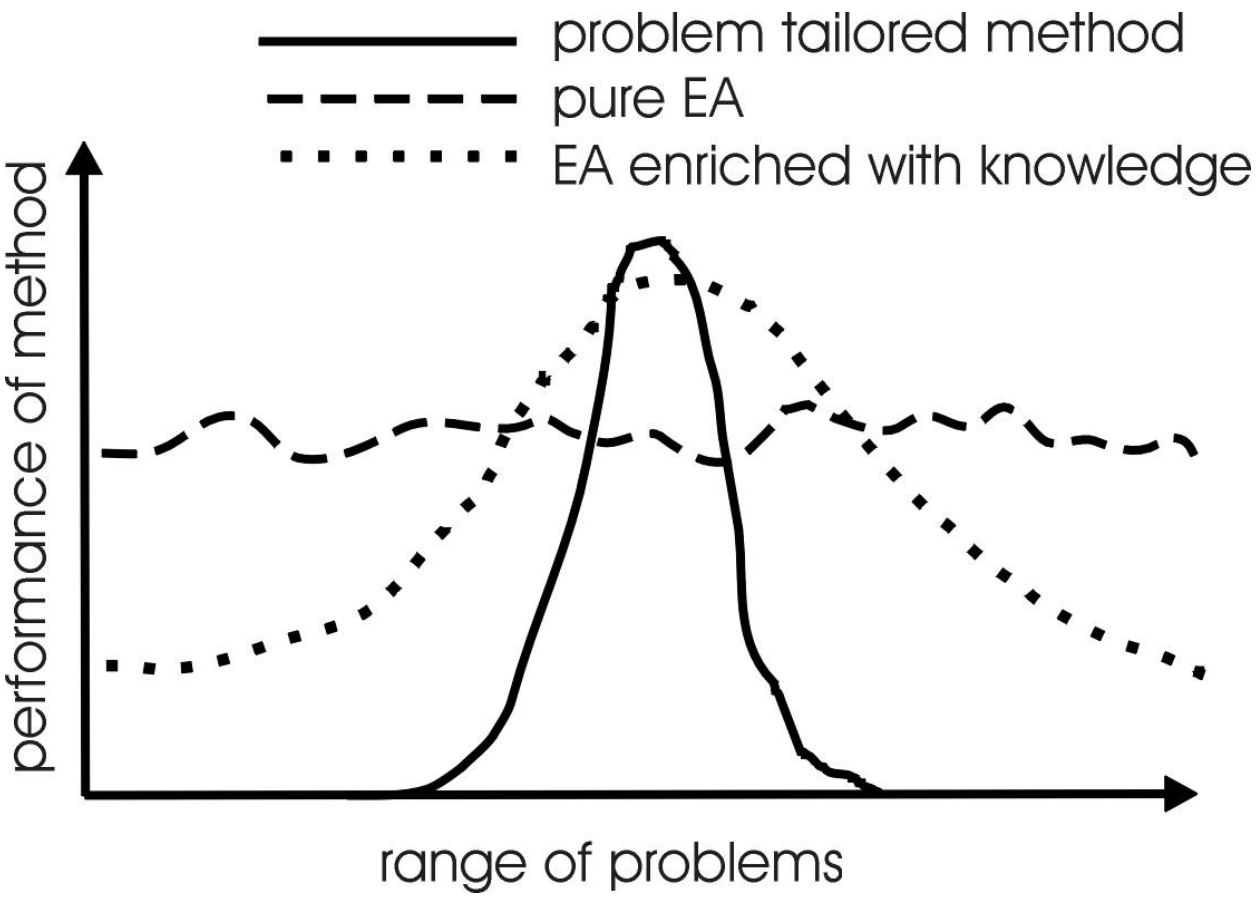
- MA are evolutionary algorithms that incorporates *global search* (crossover + mutation) and *local searches* simultaneously.

- Metaphors in natural systems:
  - Evolutionary adaptation (GA) across generations, AND
  - Individual learning within a lifetime of an individual.

- Individual learning can be implemented as (problem-specific) local searchers within MA.

Performance of EA

1990s view of EA performance after Michalewicz:



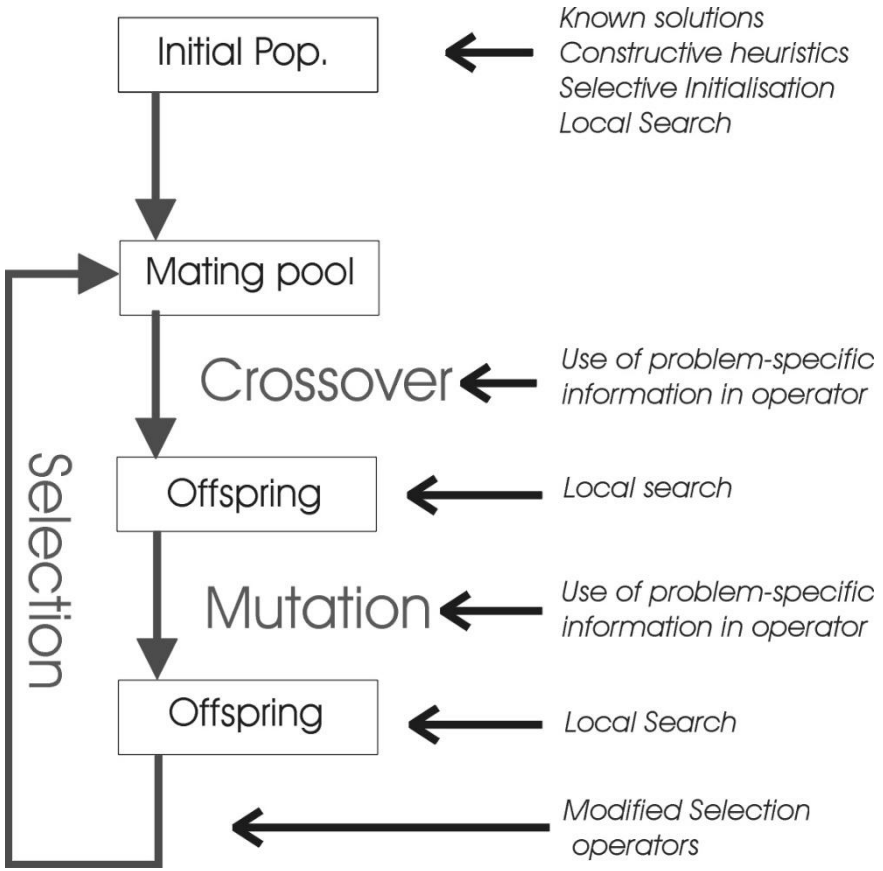
Source: Introduction to Evolutionary Computing, Eiben and Smith.

Utilizing domain knowledge

- GA can be made more *problem-specific* through the introduction of domain knowledge specific to the problem.
- Domain knowledge manifests in the forms of:
  - Specialized crossover
  - Specialized mutation
  - Problem specific representations
  - Problem specific local search
- Make EA perform better for the current tasks at hand, resulted in “Memetic algorithms”

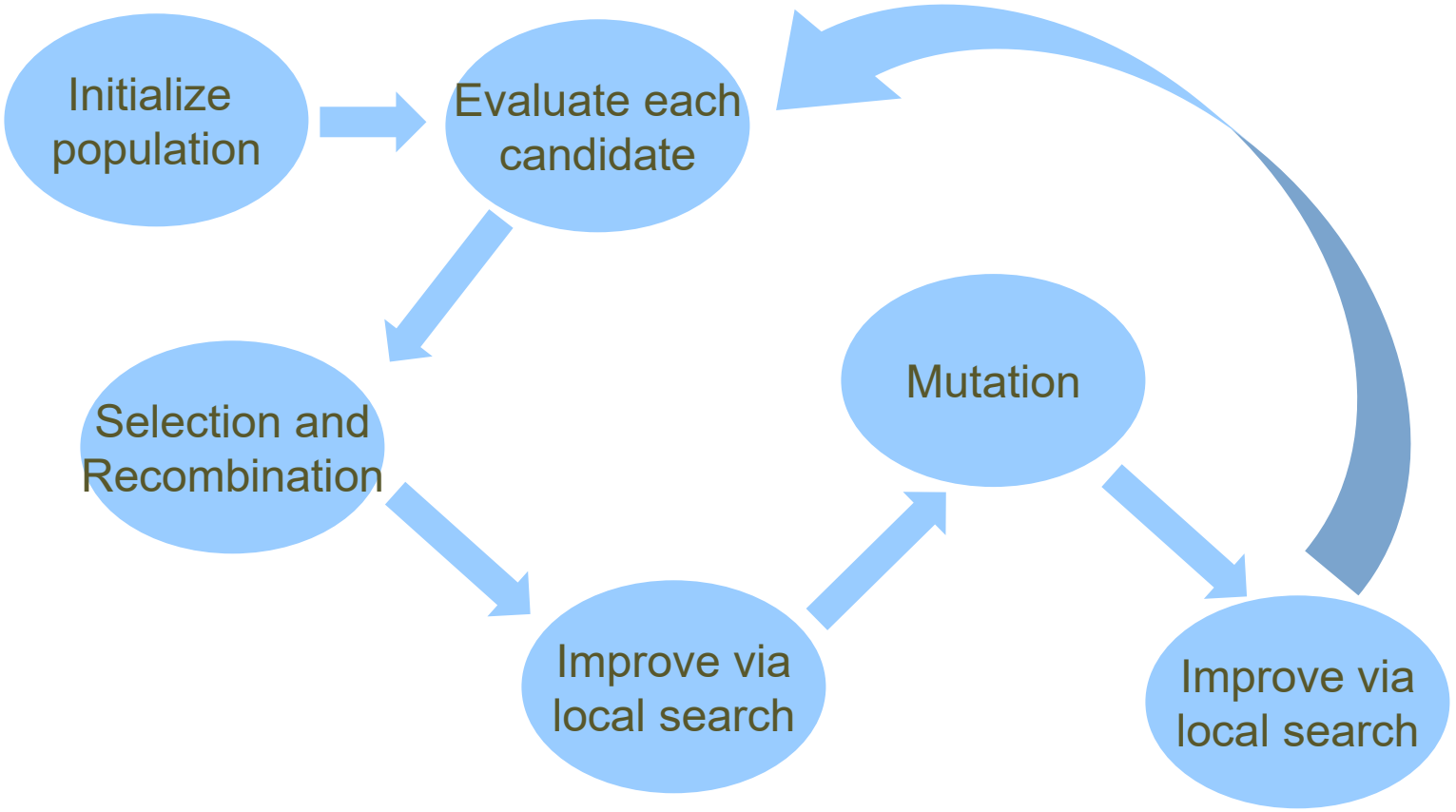
Utilizing domain knowledge

- A typical workflow of MA
- Various possibilities exist for incorporation of domain knowledge in the form of specialized evolutionary operators.



Source: Introduction to Evolutionary Computing, Eiben and Smith.

General workflow of MA



- Local search: Mechanism that enables the algorithm to obtain better solution within the neighborhood of the current individual.

Algorithm $LocalSearch(s, \mathcal{N})$

Begin

$s_{best} = s$

while (*Depth condition not satisfied*) **do**

while (*Pivot rule not satisfied*) **do**

Choose s' from the neighborhood of s, $\mathcal{N}(s)$

if ($f(s')$ is better than $f(s)$)

$s_{best} = s'$

end if

End.

$s = s_{best}$

End.

Preservation of Diversity

- The use of local search in MA can easily lead to a loss of diversity:
  - All points / individuals are moved to a local optimum
- Allow only a small fraction of the population undergo local search.
- Use multiple local searchers to induce different search space and distinct local optima
- Prevent duplicates in the selection operator
- New acceptance criteria for local search, e.g., Boltzmann method (similar to Simulated Annealing).